FINAL PROJECT PRESENTATION

Car Diagnosis Expert System using Prolog

Artificial Intelligence CCE414 | May 2026

Ramy Said
231903795

David George
231903654

Ahmed Mohamed
232903959

The Introduction

Problem Statement and Project Vision

Problem Statement

The Core Challenge

Car owners often face sudden vehicle malfunctions but lack the mechanical expertise to identify root causes.

This leads to:

- Unnecessary stress and safety hazards.
- Reliance on costly diagnostic services for simple fixes.
- Long waiting times for professional inspections.

System Purpose & Target Users

Core Purpose

Design an **expert system** to diagnose common problems via **yes/no questions**.

It identifies faults and suggests repairs efficiently for various vehicle issues.

Primary Users

Focused on **everyday car owners**, drivers, and **novice mechanics**.

Designed for individuals requiring immediate troubleshooting assistance without advanced technical knowledge.

Resources & Expertise

Software

Developed using **SWI-Prolog** for the robust knowledge base.

Knowledge

Modeled after **automotive manuals** and diagnostic trees.

Emulation

Mimics the reasoning of an **experienced auto-mechanic**.

Knowledge Engineering

Rule-Based Diagnosis Logic

Diagnostic Knowledge Base (1-5)

1. Battery
If engine **won't start** and lights are **dim**.

2. Spark Plugs
If engine **misfires** and **high fuel consumption**.

3. Radiator
If car is **overheating** and **leaking coolant**.

4. Brake Pads
If **squealing brakes** and **poor braking performance**.

5. Flat Tire
If **flapping noise** and car **pulls to side**.

Diagnostic Knowledge Base (6–10)

6. Fuel Pump
If engine **stalls** and **whining noise** exists.

7. Oxygen Sensor
If **check engine light** on and **black smoke**.

8. Alternator
If **battery light on** and **electrical issues**.

9. Transmission
If **hard shifting** and **leaking red fluid**.

10. Worn Wiper
If **poor visibility** and **streaking on glass**.

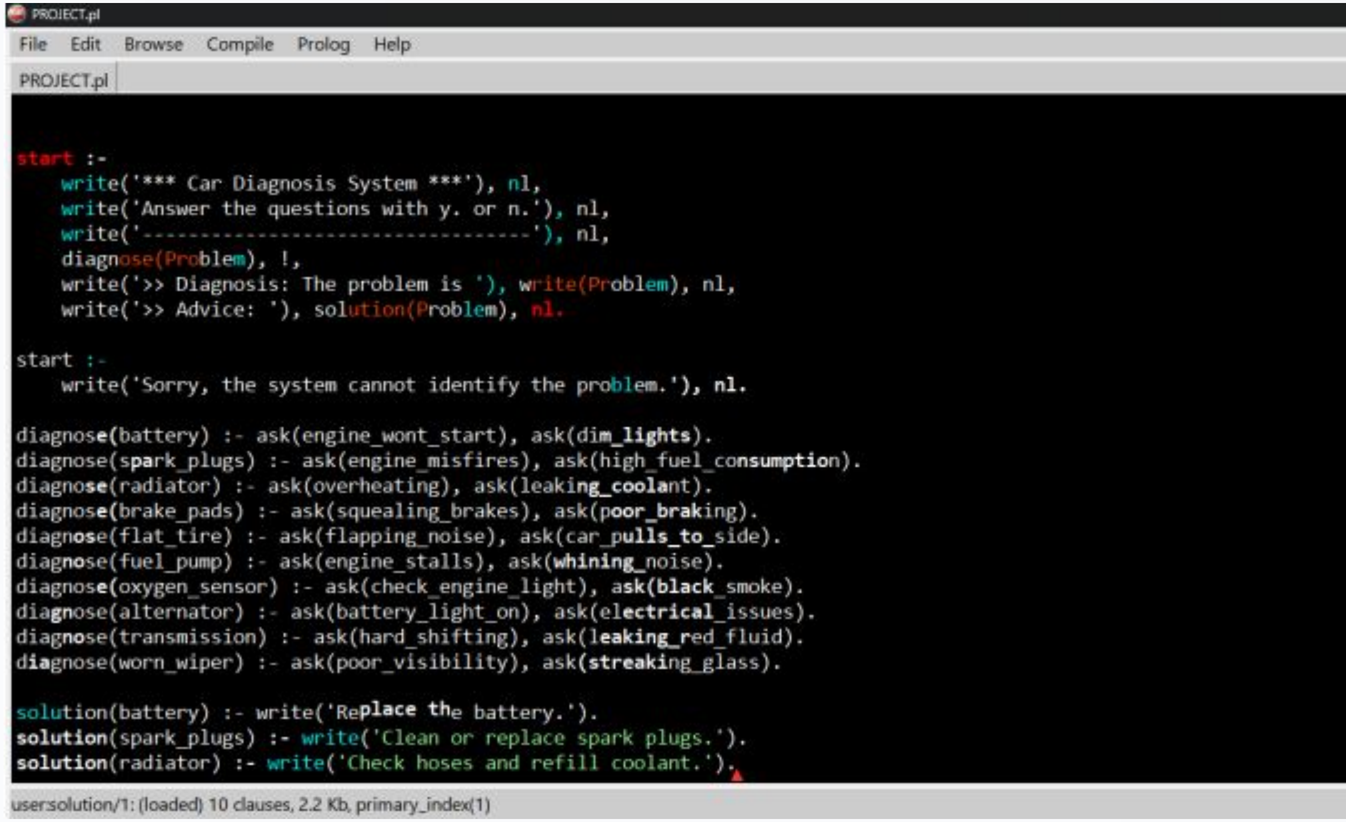
System Architecture

The system is built on **Logic Programming** using **Prolog**:

Backward Chaining
Logic travels from Goal to Facts.

The Cut Operator (!)
A crucial mechanism that prevents backtracking once a match is found.

Recursion
Used to traverse the Knowledge Base efficiently.



Diagnostic Algorithm

The algorithm follows a linear sequence from initialization to output, with a specific mechanism for handling rule matches.

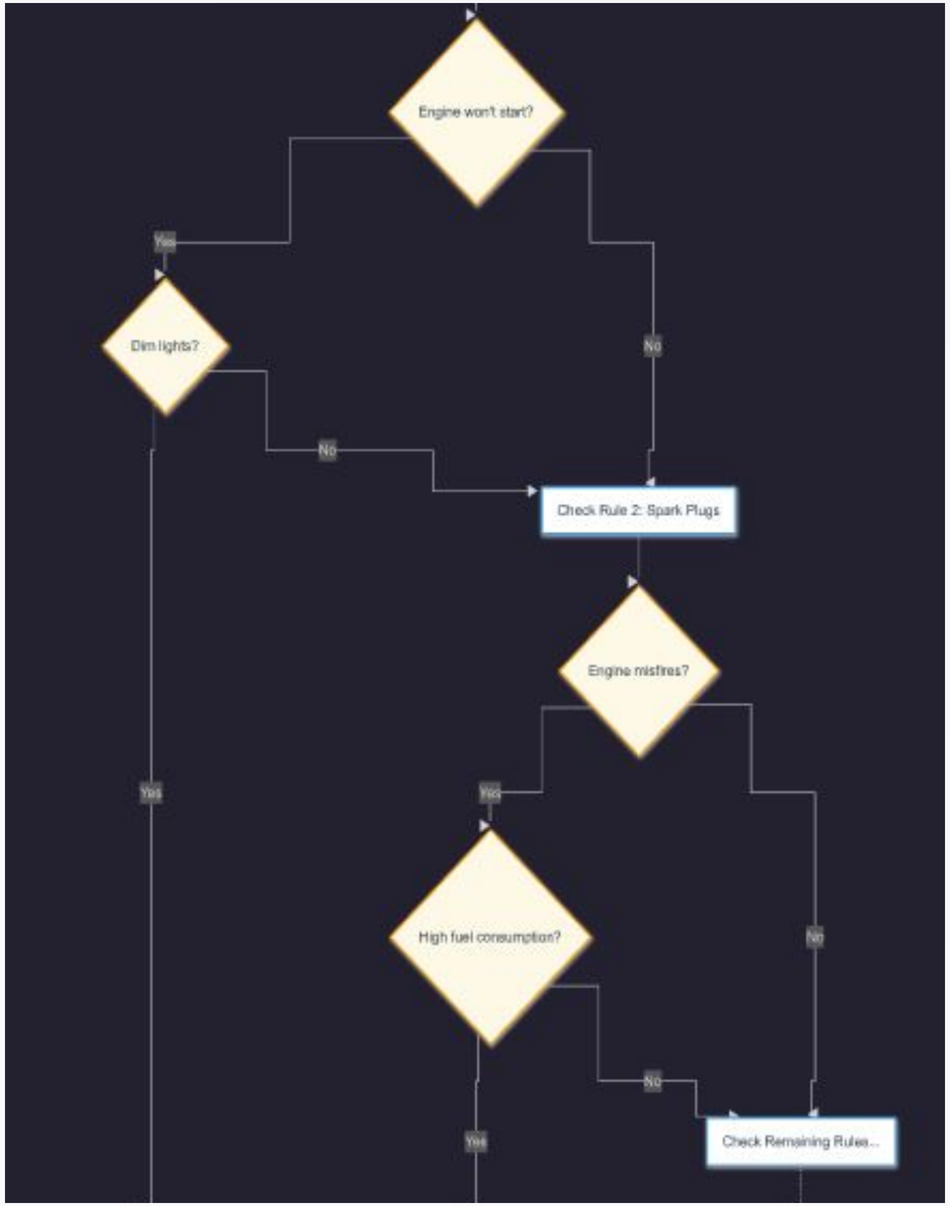
If the rule matches, the **Cut operator** stops the search. If no rules match, the **fallback predicate** informs the user.

1. Start
Initialize **start**

2. Ask
Evaluate **Diagnose Rules**

3. Process
User Input **(y/n)**

4. Output
Print Advice



System Output & Results

Diagnosis Result	User Symptoms Provided	Recommended Advice
Battery	Engine won't start, Dim lights	**Replace** the battery
Spark Plugs	Engine misfires, High consumption	**Clean/Replace** spark plugs
Radiator	Overheating, Leaking coolant	**Check** hoses/refill coolant
Unsuccessful	No matching symptoms	**System cannot identify**

System Output & Results

Diagnosis Result	User Symptoms Provided	Recommended Advice
Battery	Engine won't start; Dim lights	**Replace the battery**
Spark Plugs	Engine misfires; High consumption	**Clean/Replace spark plugs**
Radiator	Overheating; Leaking coolant	**Check hoses/refill coolant**
Unsuccessful	No matching symptoms	***System cannot identify***

Thank You!

Questions?

Artificial Intelligence | Engineering CCE414

5 / 10 / 2026